

PHASE 2 DOCUMENTATION REWRITE

Windows 98 Portfolio OS

Algorithms and Patterns — Explained From Current Code

Document	Algorithms and Patterns — Explained From Current Code
System	Windows 98 Portfolio OS
Generated	June 20, 2026 — from Markdown and repository source
Source of truth	Repository source files are authoritative; Markdown docs are background reference where details differ.
Page geometry	US Letter, 1 inch margins, repeating table header rows.

This document explains the algorithms and recurring engineering patterns in the current app. Markdown docs are used as background, but source files are treated as the authority when details differ.

1 Pattern Map

The codebase is built around a small set of recurring patterns: a reducer-owned OS state, pure subsystem modules, effect descriptors, immutable filesystem transformations, path-based window identity, lazy app loading, and browser APIs used directly instead of wrapped behind extra libraries.

Pattern	Where it appears	Why it matters
Functional core, imperative shell	src/os modules return data; store and apps perform effects	Logic is testable without the DOM.
Reducer state machine	store.tsx Action switch and OsPhase transitions	Every OS transition is explicit and inspectable.
Effect descriptors	commands.ts returns CommandEffect arrays and streams	Terminal command logic stays pure while still opening apps, mutating FS, or restarting.
Path-keyed identity	filesystem nodes and window instance IDs	Folders/files/windows dedupe by stable normalized paths.
Iterative flood algorithms	Paint and Minesweeper	No recursion depth limits; explicit stack and visited state.
Hand-written binary encoders	Paint BMP and audio WAV	The app writes real retro file formats instead of faking exports.
Direct browser APIs	Canvas, Web Audio, MediaRecorder, iframe, localStorage	Few dependencies and high transparency.

2 State Machine and Reducer Pattern

The OS is a state machine backed by one reducer. The reducer owns core transitions such as opening/focusing windows, entering BIOS setup, finishing boot, crashing, shutting down, and resetting. Provider methods read `stateRef.current`, dispatch reducer actions, and run side effects through `useEffect`.

State slice	Algorithmic role
phase / bootTarget / bootMode	Controls whether App renders boot, BIOS, desktop, DOS-only, recovery, crash, or shutdown.
windows / activeWindowId / zCounter	Models stacking, focus, minimizing, maximizing, closing, and idempotent window creation.
fs	Virtual drive state passed through pure filesystem transformations.
network	Simulated link state and ping counters used by commands and Network app.
theme/wallpaper/cursor/audio	State changes are translated into DOM/audio effects by provider useEffects.
desktopIcons and bios	Persisted user customization under win98-portfolio.v4.

src/os/store.tsx *boot target routing inside FINISH_BOOT*

```
case 'FINISH_BOOT': {
  if (state.phase === 'boot') return state
  switch (state.bootTarget) {
    case 'normal':
      if (!isSystemHealthy(state.fs)) {
        return { ...state, phase: 'bootMenu', bootTarget: 'normal' }
      }
      return { ...state, phase: 'desktop', bootMode: 'normal' }
    case 'safe':
      if (shouldSafeModeBlueScreen(state.fs)) {
        return {
          ...state,
          phase: 'crashed',
          bootMode: 'safe',
          crash: safeModeCrash(missingRequiredSystemFiles(state.fs)),
        }
      }
      return {
        ...state,
        phase: 'desktop',
        bootMode: 'safe',
        network: releasedNetworkState(),
      }
    case 'dos':
      return { ...state, phase: 'dosOnly' }
    case 'recovery':
      return { ...state, phase: 'recovery' }
  }
  return state
}
```

3 Command Engine and Effect Descriptors

The terminal command processor is the cleanest example of the functional-core pattern. `executeCommand` receives a `CommandContext` and returns `CommandOutput`. It does not open React windows or edit the store directly; it returns lines, optional stream chunks, and effect descriptors that `TerminalApp` applies through `useOs`.

Command output part	Meaning
lines	Immediate text appended to the terminal buffer.
newCwd	New working directory after <code>cd/chdir</code> .
clear	Signal to clear terminal output.
effects	Descriptions such as <code>openApp</code> , <code>setFs</code> , <code>crash</code> , <code>restart</code> , <code>setNetwork</code> , <code>exitWindow</code> .
stream	Delayed chunks used by <code>ping</code> , <code>scanreg</code> , <code>sfc</code> , and <code>scandisk</code> -style progress.

Current command switch includes 42 command labels, including aliases such as `attrib`, `calc`, `cd`, `chdir`, `chkdsk`, `cls`, `copy`, `date`, `del`, `dir`, `echo`, `erase`, `exit`, `format`, `help`, `iexplore`, `ipconfig`, `kodaking` ...

`src/os/commands.ts` *output/effect contract and streaming examples*

```
export type CommandEffect =
  | { type: 'openApp'; appId: AppId; payload?: WindowPayload }
  | { type: 'setFs'; fs: FsState }
  | { type: 'crash'; criticalPath: string }
  | { type: 'restart'; target: 'normal' | 'safe' | 'dos' | 'recovery' | 'bootMenu' }
  | { type: 'networkPing'; sent: number; received: number }
  | { type: 'setNetwork'; network: NetworkState }
  | { type: 'exitWindow' } // 'exit' closes the terminal window / leaves dosOnly to bootMenu

export type CommandContext = {
  cwd: string
  fs: FsState
  network: NetworkState
  bootMode: BootMode
  dosOnly: boolean // true when running in Command-Prompt-Only boot
}

export type CommandOutput = {
  lines: string[] // immediate output lines
  newCwd?: string
  clear?: boolean
  effects?: CommandEffect[]
  stream?: Array<{ delayMs: number; lines: string[]; effects?: CommandEffect[] }>
}

function pingCommand(args: string[], ctx: CommandContext): CommandOutput {
  const host = args.find((arg) => !arg.startsWith('/') && !arg.startsWith('-'))
  if (!host) {
    return {
      lines: [
        'Usage: ping <host>',
        '',
        'Known hosts on this network include portfolio.local, localhost,',
        'google.com, youtube.com and github.com.',
      ],
    }
  }
}
```

```

    }
  }
  const report = pingReport(host, ctx.network)
  const ip = resolveHostIp(host, ctx.network)
  if (!ip) {
    return { lines: report.lines[0] ?? [`Unknown host ${host}.`] }
  }
  const received = report.success ? 4 : 0
  const stream = report.lines.map((lines, index) => ({
    delayMs: 600,
    lines,
    effects:
      index === report.lines.length - 1
        ? ([{ type: 'networkPing', sent: 4, received } satisfies CommandEffect])
        : undefined,
  })))
  return {
    lines: [`Pinging ${host.toLowerCase()} [${ip}] with 32 bytes of data:`, ''],
    stream,
  }

function scanregCommand(args: string[], ctx: CommandContext): CommandOutput {
  const flag = args[0]?.toLowerCase()
  if (flag !== '/restore' && flag !== '/fix') {
    return { lines: scanregLines([]) }
  }
  const { fs, restored } = restoreSystemFiles(ctx.fs)
  return {
    lines: ['Microsoft Registry Checker', ''],
    stream: [
      { delayMs: 600, lines: ['Scanning system registry...'] },
      { delayMs: 700, lines: ['Checking registry backups: rb000.cab rb001.cab rb002.cab'] },
      { delayMs: 800, lines: [restored.length ? `Found ${restored.length} damaged or missing` : ''] },
      { delayMs: 900, lines: scanregLines(restored), effects: [{ type: 'setFs', fs }] },
    ],
  }
}

```

```
function sfcCommand(args: string[], ctx: CommandContext): CommandOutput {
  const flag = args[0]?.toLowerCase()
  if (flag !== '/scannow') {
    return { lines: ['Checks the integrity of protected system files.', '', 'Usage: sfc /sca
  ]
  }
  const { fs, restored } = restoreSystemFiles(ctx.fs)
  return {
    lines: ['System File Checker', '', ''],
    stream: [
      { delayMs: 700, lines: ['Verification 25% complete.'] },
      { delayMs: 700, lines: ['Verification 50% complete.'] },
      { delayMs: 700, lines: ['Verification 75% complete.'] },
      { delayMs: 900, lines: sfcLines(restored), effects: [{ type: 'setFs', fs }] },
    ],
  }
}
```

TerminalApp owns the imperative side of those descriptors. That split is why commands can be unit-tested as data transformations, while the app can still perform UI effects like opening windows, closing the DOS-only terminal, or replacing the filesystem.

`src/components/apps/TerminalApp.tsx`*applying command effects*

```
function applyEffects(effects: CommandEffect[] | undefined) {
  effects?.forEach((effect) => {
    switch (effect.type) {
      case 'openApp':
        openApp(effect.appId, effect.payload)
        break
      case 'setFs':
        fsOps.replaceFs(effect.fs)
        break
      case 'crash':
        crashSystem(crashFor(effect.criticalPath))
        break
      case 'restart':
        restart(effect.target, { bootProfile: 'warm' })
        break
      case 'networkPing':
        networkOps.recordTraffic(effect.sent, effect.received)
        break
      case 'setNetwork':
        networkOps.applyConfig(effect.network)
        break
      case 'exitWindow':
        if (dosOnly) {
          restart('bootMenu')
        } else {
          closeWindow(windowId)
        }
        break
    }
  })
}

function runCommand(command: string) {
  const shownPrompt = `${prompt}${command}`
}
```

```
const output = executeCommand(command, {
  cwd,
  fs: state.fs,
  network: state.network,
  bootMode: state.bootMode,
  dosOnly,
})

if (output.clear) {
  setLines([])
} else {
  append([shownPrompt, ...output.lines])
}

if (output.newCwd) {
  setCwd(output.newCwd)
}

applyEffects(output.effects)

let totalDelay = 0
output.stream?.forEach((chunk) => {
  totalDelay += chunk.delayMs
  const timer = window.setTimeout(() => {
    append(chunk.lines)
    applyEffects(chunk.effects)
  }, totalDelay)
  timersRef.current.push(timer)
})
```

4 Virtual Filesystem Algorithms

The filesystem stores a tree as a flat path-keyed map. This makes lookup cheap and makes path rewrites explicit. Folder rename/move is the interesting case: every descendant key and every child path must be rewritten with the new root prefix.

[src/os/filesystem.ts](#) *subtree re-keying*

```
function rekeySubtree(fs: FsState, oldRoot: string, newRoot: string): Record<string, FsNode>
  const subtree = collectSubtree(fs, oldRoot)
  const next: Record<string, FsNode> = {}
  const rename = (path: string) => `${newRoot}${path.slice(oldRoot.length)}`
  for (const [path, node] of Object.entries(subtree)) {
    const newPath = rename(path)
    next[newPath] = {
      ...node,
      path: newPath,
      name: newPath === newRoot ? baseName(newRoot) : node.name,
      children: node.children?.map(rename),
    }
  }
  return next
}
```

Operation	Algorithmic detail
normalizePath / resolvePath	Canonicalizes slashes, drive defaults, relative segments, and case-insensitive lookup fallback.
collectSubtree	Gathers folder descendants before delete/copy/move operations.
rekeySubtree	Uses string-prefix substitution to rename every descendant path and child reference.
deleteNode	Creates a RecycleEntry containing original paths, removes nodes, and reports criticalDeleted.
restoreEntry	Recreates missing parents, reinserts original nodes, and repairs parent children arrays.
openTargetFor	Maps folders/files/launcher nodes to app IDs and payloads.

`src/os/filesystem.ts` *delete and association outputs*

```
export function deleteNode(fs: FsState, path: string): { fs: FsState; error: string | null;
  const node = getNode(fs, path)
  if (!node) {
    return { fs, error: 'The system cannot find the file specified.', criticalDeleted: false
  }
  const normalized = node.path
  if (/^[A-Z]:\\$/.test(normalized)) {
    return { fs, error: 'Cannot delete a drive.', criticalDeleted: false }
  }
  const subtree = collectSubtree(fs, normalized)
  const critical = isCriticalPath(normalized) || Object.keys(subtree).some((key) => isCriticalPath(normalized + key))
  recycleCounter += 1
  const entry: RecycleEntry = {
    id: `re-${Date.now()}-${recycleCounter}`,
    rootPath: normalized,
    name: node.name,
    icon: node.icon,
    fileType: node.fileType,
    deletedAt: nowStamp(),
    critical,
    nodes: subtree,
  }
  const nodes: Record<string, FsNode> = {}
  for (const [key, value] of Object.entries(fs.nodes)) {
    if (!subtree[key]) {
      nodes[key] = value
    }
  }
  let next: FsState = { ...fs, nodes, recycle: [entry, ...fs.recycle] }
  next = detachChild(next, parentPath(normalized), normalized)
  return { fs: next, error: null, criticalDeleted: critical }

export function openTargetFor(node: FsNode): { appId: AppId; payload: WindowPayload } | null {
  if (node.appId) {
```

```
    return { appId: node.appId, payload: node.appPayload ?? {} }
  }
  if (node.kind === 'folder') {
    return { appId: 'explorer', payload: { path: node.path } }
  }
  const ext = extensionOf(node.name)
  if (TEXT_EXTENSIONS.has(ext)) {
    return { appId: 'notepad', payload: { filePath: node.path } }
  }
  if (DOCUMENT_EXTENSIONS.has(ext)) {
    return { appId: 'wordpad', payload: { filePath: node.path } }
  }
  if (IMAGE_EXTENSIONS.has(ext)) {
    return { appId: 'imageViewer', payload: { filePath: node.path } }
  }
  if (AUDIO_EXTENSIONS.has(ext)) {
    return { appId: 'mediaPlayer', payload: { filePath: node.path } }
  }
  if (VIDEO_EXTENSIONS.has(ext)) {
    return { appId: 'videoPlayer', payload: { filePath: node.path } }
  }
  if (WEB_EXTENSIONS.has(ext)) {
    return { appId: 'internetExplorer', payload: { url: node.content ?? 'about:home' } }
  }
}
```

5 Recovery and Health Checks

Recovery is modeled as a comparison between current filesystem state and the initial Windows seed. Required system files define boot health. Missing files cause normal boot to fall back to the startup menu; two or more missing required files can make Safe Mode blue-screen too.

`src/os/recovery.ts` *health and restore rules*

```
export function missingSystemFiles(fs: FsState): string[] {
  return initialWindowsPaths().filter((path) => !fs.nodes[path])
}

export function isSystemHealthy(fs: FsState): boolean {
  return REQUIRED_SYSTEM_FILES.every((path) => Boolean(fs.nodes[normalizePath(path)]))
}

export function missingRequiredSystemFiles(fs: FsState): string[] {
  return REQUIRED_SYSTEM_FILES.filter((path) => !fs.nodes[normalizePath(path)])
}

export function shouldSafeModeBlueScreen(fs: FsState): boolean {
  return missingRequiredSystemFiles(fs).length ≥ 2
}

/**
 * Re-inserts every missing initial C:\Windows node and drops recycle entries
 * that pointed at the now-restored system files.
 */
export function restoreSystemFiles(fs: FsState): { fs: FsState; restored: string[] } {
  const pristine = initialFs()
  const missing = missingSystemFiles(fs)
  if (!missing.length) {
    return { fs, restored: [] }
  }

  const nodes: Record<string, FsNode> = { ...fs.nodes }
  for (const path of missing) {
    const original = pristine.nodes[path]
    if (original) {
      nodes[path] = original
    }
  }
}
```

```

// Rebuild children lists for every folder in the protected scope so restored
// entries reappear in their initial order (plus any survivors not in the
// pristine tree, which normally cannot exist because creates are denied).
for (const [path, node] of Object.entries(nodes)) {
  if (node.kind !== 'folder') continue
  const pristineNode = pristine.nodes[path]
  if (!pristineNode?.children) continue
  if (!isProtectedPath(path) && path.toLowerCase() !== 'c:\\') continue
  const current = nodes[path].children ?? []
  const fromInitial = pristineNode.children.filter((child) => Boolean(nodes[child]))
  const extras = current.filter((child) => !pristineNode.children?.includes(child) && Boolean(nodes[child]))
  nodes[path] = { ...node, children: [...fromInitial, ...extras] }
}

const recycle = fs.recycle.filter((entry) => !isProtectedPath(entry.rootPath))

return { fs: { ...fs, nodes, recycle }, restored: missing }
}

function restoredFileList(restored: string[]): string[] {
  return restored.filter((path) => initialFs().nodes[path]?.kind !== 'folder')
}

```

6 Window Identity, Z-Order, Drag, and Resize

Window management is mostly data. Each window has an `instanceId`, `rect`, `zIndex`, `minimized/maximized` flags, and optional payload. `instanceIdFor` decides whether `openApp` should create a new window or focus/update an existing one.

`src/os/store.tsx` *window identity*

```
function instanceIdFor(appId: AppId, payload?: WindowPayload): string {
  const singleton = appDefinitions[appId].singleton === false
  if (appId === 'explorer') {
    return `explorer:${normalizePath(payload?.path ?? 'C:\\').toLowerCase()}`
  }
  if (appId === 'projectDetails' && payload?.projectId) {
    return `projectDetails:${payload.projectId}`
  }
  if (appId === 'notepad' && payload?.filePath) {
    return `notepad:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (appId === 'wordpad' && payload?.filePath) {
    return `wordpad:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (appId === 'imageViewer' && payload?.filePath) {
    return `imageViewer:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (appId === 'videoPlayer' && payload?.filePath) {
    return `videoPlayer:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (singleton) {
    return appId
  }
  instanceCounter += 1
  return `${appId}#${instanceCounter}`
}
```

Focus does not reorder the array. The reducer increments `zCounter` and stamps the focused window with the newest `zIndex`, so CSS naturally puts the newest focused window on top.

`src/os/store.tsx` and `WindowFrame.tsx` *z-order and geometry*

```
case 'FOCUS_WINDOW': {
  const target = state.windows.find((win) => win.instanceId === action.id)
  if (!target) return state
  const zIndex = state.zCounter + 1
  return {
    ...state,
    zCounter: zIndex,
    activeWindowId: action.id,
    windows: state.windows.map((win) =>
      win.instanceId === action.id ? { ...win, minimized: false, zIndex } : win,
    ),
  }
}

function clampWindow(rect: WindowRect): WindowRect {
  const viewportWidth = window.innerWidth
  const viewportHeight = window.innerHeight - taskbarHeight
  const width = Math.min(Math.max(minWindowWidth, rect.width), Math.max(minWindowWidth, viewportWidth))
  const height = Math.min(Math.max(minWindowHeight, rect.height), Math.max(minWindowHeight, viewportHeight))
  return {
    width,
    height,
    x: Math.max(8, Math.min(rect.x, viewportWidth - width - 8)),
    y: Math.max(8, Math.min(rect.y, viewportHeight - height - 8)),
  }
}

function resizeWindowRect(resize: ResizeState, clientX: number, clientY: number): WindowRect {
  const dx = clientX - resize.startX
  const dy = clientY - resize.startY
  const next = { ...resize.startRect }

  if (resize.direction.includes('e')) {
    next.width = resize.startRect.width + dx
  }
}
```

```
}
if (resize.direction.includes('s')) {
  next.height = resize.startRect.height + dy
}
if (resize.direction.includes('w')) {
  next.width = resize.startRect.width - dx
  next.x = resize.startRect.x + dx
  if (next.width < minWindowWidth) {
    next.x = resize.startRect.x + resize.startRect.width - minWindowWidth
    next.width = minWindowWidth
  }
}
if (resize.direction.includes('n')) {
  next.height = resize.startRect.height - dy
  next.y = resize.startRect.y + dy
  if (next.height < minWindowHeight) {
    next.y = resize.startRect.y + resize.startRect.height - minWindowHeight
    next.height = minWindowHeight
  }
}

return clampWindow(next)
}

export function WindowFrame({
  window: windowState,
```

7 Paint Algorithms

Paint contains two real algorithms: flood fill over the canvas pixel buffer and BMP export. The flood fill is stack-based and uses a Uint8Array visited map. The BMP encoder writes a true 24-bit uncompressed BMP with the required bottom-up BGR rows and 4-byte row padding.

[src/components/apps/PaintApp.tsx](#) *BMP encoder and flood fill*

```
function canvasToBmpDataUrl(canvas: HTMLCanvasElement): string {
  const ctx = canvas.getContext('2d')
  if (!ctx) return canvas.toDataURL('image/png')
  const { width, height } = canvas
  const { data } = ctx.getImageData(0, 0, width, height)
  const rowSize = Math.floor((24 * width + 31) / 32) * 4
  const pixelArraySize = rowSize * height
  const fileSize = 54 + pixelArraySize
  const buffer = new ArrayBuffer(fileSize)
  const view = new DataView(buffer)
  view.setUint8(0, 0x42) // 'B'
  view.setUint8(1, 0x4d) // 'M'
  view.setUint32(2, fileSize, true)
  view.setUint32(10, 54, true) // pixel data offset
  view.setUint32(14, 40, true) // DIB header size
  view.setInt32(18, width, true)
  view.setInt32(22, height, true) // positive height = bottom-up rows
  view.setUint16(26, 1, true) // planes
  view.setUint16(28, 24, true) // bits per pixel
  view.setUint32(34, pixelArraySize, true)
  view.setInt32(38, 2835, true) // ~72 DPI
  view.setInt32(42, 2835, true)
  let offset = 54
  for (let y = height - 1; y ≥ 0; y -= 1) {
    for (let x = 0; x < width; x += 1) {
      const i = (y * width + x) * 4
      view.setUint8(offset, data[i + 2]) // B
      view.setUint8(offset + 1, data[i + 1]) // G
      view.setUint8(offset + 2, data[i]) // R
      offset += 3
    }
    for (let pad = width * 3; pad < rowSize; pad += 1) {
      view.setUint8(offset, 0)
      offset += 1
    }
  }
}
```

```

    }
  }
  const bytes = new Uint8Array(buffer)
  let binary = ''
  const chunk = 8192
  for (let i = 0; i < bytes.length; i += chunk) {
    binary += String.fromCharCode(...bytes.subarray(i, i + chunk))
  }
  return `data:image/bmp;base64,${btoa(binary)}`
}

function hexToRgb(hex: string): [number, number, number] {

function floodFill(ctx: CanvasRenderingContext2D, width: number, height: number, sx: number,
  const xi = Math.max(0, Math.min(width - 1, Math.floor(sx)))
  const yi = Math.max(0, Math.min(height - 1, Math.floor(sy)))
  const img = ctx.getImageData(0, 0, width, height)
  const d = img.data
  const start = (yi * width + xi) * 4
  const tr = d[start], tg = d[start + 1], tb = d[start + 2], ta = d[start + 3]
  const [fr, fg, fb] = hexToRgb(hex)
  if (Math.abs(fr - tr) ≤ 2 && Math.abs(fg - tg) ≤ 2 && Math.abs(fb - tb) ≤ 2 && ta ≡ 2
  // Tight tolerance so the fill stops at a drawn shape's outline instead of
  // bleeding across it – clicking inside a circle fills only that circle.
  const tol = 20
  const seen = new Uint8Array(width * height)
  const stack = [yi * width + xi]
  while (stack.length) {
    const p = stack.pop() as number
    if (seen[p]) continue
    seen[p] = 1
    const i = p * 4
    if (
      Math.abs(d[i] - tr) > tol ||
      Math.abs(d[i + 1] - tg) > tol ||
      Math.abs(d[i + 2] - tb) > tol ||

```

```
    Math.abs(d[i + 3] - ta) > tol
  ) {
    continue
  }
  d[i] = fr; d[i + 1] = fg; d[i + 2] = fb; d[i + 3] = 255
  const x = p % width
  const y = (p - x) / width
  if (x + 1 < width) stack.push(p + 1)
  if (x - 1 ≥ 0) stack.push(p - 1)
  if (y + 1 < height) stack.push(p + width)
  if (y - 1 ≥ 0) stack.push(p - width)
}
ctx.putImageData(img, 0, 0)
}

export function PaintApp({ windowId, payload }: AppProps) {
  const { state, fsOps, setWindowTitle, showMessageBox } = useOs()
```

8 Minesweeper Algorithms

Minesweeper also uses explicit grid algorithms. `neighbors` returns valid surrounding cells. `placeMines` waits until the first reveal and forbids both the clicked cell and its neighbors, so the first click opens safely. `floodReveal` iteratively reveals connected empty cells.

[src/components/apps/MinesweeperApp.tsx](#) *mine placement and reveal*

```
function neighbors(r: number, c: number, rows: number, cols: number): Array<[number, number]> {
  const result: Array<[number, number]> = []
  for (let dr = -1; dr ≤ 1; dr += 1) {
    for (let dc = -1; dc ≤ 1; dc += 1) {
      if (dr === 0 && dc === 0) continue
      const nr = r + dr
      const nc = c + dc
      if (nr ≥ 0 && nr < rows && nc ≥ 0 && nc < cols) result.push([nr, nc])
    }
  }
  return result
}

/** Place mines after the first click so the first reveal is always safe (and opens an area)
function placeMines(board: Cell[][], mines: number, safeR: number, safeC: number): Cell[][] {
  const rows = board.length
  const cols = board[0].length
  const next = board.map((row) => row.map((cell) => ({ ...cell, mine: false, adjacent: 0 })))
  const forbidden = new Set<string>([`${safeR},${safeC}`, ...neighbors(safeR, safeC, rows, cols)])
  let placed = 0
  let guard = 0
  while (placed < mines && guard < 100000) {
    guard += 1
    const r = Math.floor(Math.random() * rows)
    const c = Math.floor(Math.random() * cols)
    if (forbidden.has(`${r},${c}`) || next[r][c].mine) continue
    next[r][c].mine = true
    placed += 1
  }
  for (let r = 0; r < rows; r += 1) {
    for (let c = 0; c < cols; c += 1) {
      if (next[r][c].mine) continue
      next[r][c].adjacent = neighbors(r, c, rows, cols).filter(([nr, nc]) => next[nr][nc].mine).length
    }
  }
}
```

```
}
return next
}

function floodReveal(board: Cell[][], startR: number, startC: number): Cell[][] {
  const rows = board.length
  const cols = board[0].length
  const next = board.map((row) => row.map((cell) => ({ ...cell })))
  const stack: Array<[number, number]> = [[startR, startC]]
  while (stack.length) {
    const [r, c] = stack.pop() as [number, number]
    const cell = next[r][c]
    if (cell.revealed || cell.flagged) continue
    cell.revealed = true
    if (cell.adjacent === 0 && !cell.mine) {
      for (const [nr, nc] of neighbors(r, c, rows, cols)) {
        if (!next[nr][nc].revealed) stack.push([nr, nc])
      }
    }
  }
  return next
}
```

9 Calculator Expression Evaluation

Calculator deliberately avoids a full parser. It strips the expression to a tiny arithmetic whitelist, then uses JavaScript's native precedence through Function in strict mode. The result is rounded to six decimal places to hide floating-point noise, while invalid or non-finite results become Error.

`src/components/apps/CalculatorApp.tsx` *calculate*

```
function calculate(expression: string) {
  const safe = expression.replace(/^[^0-9+\-*/.() ]/g, '')
  if (!safe.trim()) {
    return '0'
  }
  try {
    const result = Function(`"use strict"; return (${safe})`)() as number
    return Number.isFinite(result) ? String(Math.round(result * 1000000) / 1000000) : 'Error'
  } catch {
    return 'Error'
  }
}
```

10 Audio Synthesis and WAV Encoding

The audio system is not a set of sample files. It schedules oscillator tones and noise envelopes through Web Audio. The same `scheduleSound` recipes can play live through `AudioContext` or render offline through `OfflineAudioContext`. `encodeWav` then writes a PCM WAV by hand.

Audio layer	What it does
tone / noise helpers	Schedule oscillator or noise nodes with gain envelopes.
<code>scheduleSound</code>	Maps <code>SoundId</code> to short retro sound recipes and returns duration.
<code>unlockAudio</code> / <code>playSound</code>	Handles browser gesture restrictions and live playback.
<code>renderSoundToWavDataUrl</code>	Renders a synth sound offline into a data URL.
<code>encodeWav</code>	Writes RIFF/WAVE/fmt/data chunks and 16-bit PCM samples with <code>DataView</code> .

[src/os/audio.ts](#) *scheduling and WAV encoding*

```
export function scheduleSound(ctx: BaseAudioContext, dest: AudioNode, id: SoundId, volume: number) {
  const master = Math.max(0, Math.min(1, volume))
  switch (id) {
    case 'startup': {
      // Warm pad chord rising into a soft bell – original jingle.
      const chord = [220, 277.18, 329.63, 440]
      chord.forEach((freq, index) => {
        tone(ctx, dest, master, {
          type: 'triangle',
          freq,
          start: index * 0.05,
          duration: 2.3,
          peak: 0.12,
          attack: 0.9,
        })
      })
      tone(ctx, dest, master, { type: 'sine', freq: 880, start: 1.1, duration: 1.4, peak: 0.1 })
      tone(ctx, dest, master, { type: 'sine', freq: 1108.73, start: 1.45, duration: 1.1, peak: 0.1 })
      noise(ctx, dest, master, { start: 0, duration: 1.2, peak: 0.015, filterType: 'lowpass' })
      break
    }
    case 'shutdown': {
      // Gentle descending farewell.
      const steps = [523.25, 392, 329.63, 261.63]
      steps.forEach((freq, index) => {
        tone(ctx, dest, master, {
          type: 'triangle',
          freq,
          start: index * 0.28,
          duration: 0.8,
          peak: 0.14,
          attack: 0.06,
        })
      })
    }
  }
}
```

```

        tone(ctx, dest, master, { type: 'sine', freq: 130.81, start: 0.84, duration: 1.0, peak: 0.2 })
        break
    }
    case 'error': {
        tone(ctx, dest, master, { type: 'square', freq: 196, endFreq: 130, start: 0, duration: 0.3, peak: 0.2 })
        tone(ctx, dest, master, { type: 'square', freq: 98, start: 0, duration: 0.3, peak: 0.2 })
        break
    }
    case 'warn': {
        tone(ctx, dest, master, { type: 'square', freq: 392, start: 0, duration: 0.16, peak: 0.2 })
        tone(ctx, dest, master, { type: 'square', freq: 311.13, start: 0.2, duration: 0.2, peak: 0.2 })
        break
    }
    case 'click': {
        noise(ctx, dest, master, { start: 0, duration: 0.018, peak: 0.2, filterType: 'highpass' })
        tone(ctx, dest, master, { type: 'square', freq: 1800, start: 0, duration: 0.012, peak: 0.2 })
    }
}

function encodeWav(buffer: AudioBuffer): string {
    const samples = buffer.getChannelData(0)
    const sampleRate = buffer.sampleRate
    const bytesPerSample = 2
    const dataSize = samples.length * bytesPerSample
    const arrayBuffer = new ArrayBuffer(44 + dataSize)
    const view = new DataView(arrayBuffer)

    const writeString = (offset: number, text: string) => {
        for (let i = 0; i < text.length; i += 1) {
            view.setUint8(offset + i, text.charCodeAt(i))
        }
    }

    writeString(0, 'RIFF')
    view.setUint32(4, 36 + dataSize, true)
    writeString(8, 'WAVE')
    writeString(12, 'fmt ')
    view.setUint32(16, 16, true) // PCM chunk size

```

```
view.setUint16(20, 1, true) // PCM format
view.setUint16(22, 1, true) // mono
view.setUint32(24, sampleRate, true)
view.setUint32(28, sampleRate * bytesPerSample, true) // byte rate
view.setUint16(32, bytesPerSample, true) // block align
view.setUint16(34, 16, true) // bits per sample
writeString(36, 'data')
view.setUint32(40, dataSize, true)

let offset = 44
for (let i = 0; i < samples.length; i += 1) {
  const clamped = Math.max(-1, Math.min(1, samples[i]))
  view.setInt16(offset, clamped < 0 ? clamped * 0x8000 : clamped * 0x7fff, true)
  offset += 2
}

// Base64-encode in chunks to avoid call-stack limits.
const bytes = new Uint8Array(arrayBuffer)
let binary = ''
const chunk = 8192
for (let i = 0; i < bytes.length; i += chunk) {
  binary += String.fromCharCode(...bytes.subarray(i, i + chunk))
}
```

11 Network Simulation

Network state is deterministic enough to feel like a small LAN but random enough to feel alive. Known hosts resolve to fixed IPs, DHCP leases pick an address on 192.168.98.x, offline links time out, and reachable pings stream four replies plus statistics.

src/os/network.ts *host resolution and ping report*

```
    connectedSince: '06/12/2026 12:00 AM',
  }

  /** A fresh, connected DHCP lease on the 192.168.98.x subnet. */
  export function randomDhcpLease(): NetworkState {
    const lastOctet = 20 + Math.floor(Math.random() * 230)
    return {
      connected: true,
      dhcp: true,
      adapterName: ADAPTER_NAME,
      macAddress: MAC_ADDRESS,
      ipAddress: `192.168.98.${lastOctet}`,
      subnetMask: SUBNET,
      gateway: GATEWAY,
      dns: DNS,
      packetsSent: 0,
      packetsReceived: 0,
      connectedSince: nowStamp(),
    }
  }

  /** The adapter after `ipconfig /release` (or unplugging the cable). */
  export function releasedNetworkState(): NetworkState {
    return {
      connected: false,
      dhcp: true,
      adapterName: ADAPTER_NAME,
      macAddress: MAC_ADDRESS,
      ipAddress: '0.0.0.0',

      if (colon ≥ 0) host = host.slice(0, colon)
      return host
    }
  }
```

```
/** Resolves a host to a simulated IP, or null when unknown. */
export function resolveHostIp(hostOrUrl: string, network: NetworkState): string | null {
  const host = hostOf(hostOrUrl)
  if (!host) return null
  if (IP_RE.test(host)) return host
  if (host === 'localhost') return '127.0.0.1'
  if (KNOWN_HOSTS[host]) return KNOWN_HOSTS[host]
  if (network.gateway && host === network.gateway) return network.gateway
  if (network.dns && host === network.dns) return network.dns
  return null
}

/** True when the host resolves and the simulated link can carry packets to it. */
export function resolveHostReachable(hostOrUrl: string, network: NetworkState): boolean {
  const ip = resolveHostIp(hostOrUrl, network)
  if (!ip) return false
  if (ip === '127.0.0.1' || ip === network.ipAddress) {
    return true // loopback always answers
  }
  return network.connected
}

export type PingReport = {
  /** Groups of lines to stream: 4 reply lines, then a blank line + statistics. */
  lines: string[][]
  success: boolean
}

export function pingReport(host: string, network: NetworkState): PingReport {
  const ip = resolveHostIp(host, network)
  if (!ip) {
    return { lines: [['Unknown host ${hostOf(host) || host.trim()}'], success: false ] }
  }
  const reachable = resolveHostReachable(host, network)
  if (!reachable) {
    return {
```

```

    lines: [
      ['Request timed out.'],
      ['Request timed out.'],
      ['Request timed out.'],
      ['Request timed out.'],
      [
        '',
        `Ping statistics for ${ip}:`,
        '    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),',
      ],
    ],
    success: false,
  }
}

const local = ip === '127.0.0.1' || ip === network.ipAddress || ip === network.gateway
const times: number[] = []
const replies = [0, 1, 2, 3].map(() => {
  const time = local ? 1 + Math.floor(Math.random() * 2) : 12 + Math.floor(Math.random() *
times.push(time)
  const shown = time ≤ 1 ? 'time<1ms' : `time=${time}ms`
  return [`Reply from ${ip}: bytes=32 ${shown} TTL=${local ? 128 : 53}`]
})

const min = Math.min(...times)
const max = Math.max(...times)
const avg = Math.round(times.reduce((sum, value) => sum + value, 0) / times.length)
return {
  lines: [
    ...replies,
    [
      '',
      `Ping statistics for ${ip}:`,

```

12 Persistence and Source Corrections

Some Markdown still mentions older persistence keys. The current code is version 4 and can migrate v3. The persisted snapshot includes filesystem, theme, wallpaper, cursor, audio, network, desktop icon positions, and BIOS settings.

[src/os/persistence.ts](#) *current persistence version*

```
import type { AudioState, BiosSettings, CursorSchemeId, FsState, NetworkState, OsState, Point } from '../types'
import { defaultBiosSettings, normalizeBootOrder } from '../data/bios'

export type PersistedState = {
  version: 4
  fs: FsState
  themeId: string
  wallpaperId: string
  cursorScheme: CursorSchemeId
  audio: AudioState
  network: NetworkState
  desktopIcons: Record<string, Point>
  bios: BiosSettings
}

const STORAGE_KEY = 'win98-portfolio.v4'
const LEGACY_STORAGE_KEY = 'win98-portfolio.v3'

export function persistState(state: OsState): void {
  try {
    if (typeof localStorage === 'undefined') {
      return
    }
    const snapshot: PersistedState = {
      version: 4,
      fs: {
        nodes: state.fs.nodes,
        // onResult callbacks etc. never live in fs, so it serializes cleanly -
        // but strip functions defensively by relying on JSON semantics.
        recycle: state.fs.recycle,
      },
      themeId: state.themeId,
      wallpaperId: state.wallpaperId,
```

```

    cursorScheme: state.cursorScheme,
    audio: state.audio,
    network: state.network,
    desktopIcons: state.desktopIcons,
    bios: state.bios,
  }
  localStorage.setItem(STORAGE_KEY, JSON.stringify(snapshot))
} catch {

```

13 Accuracy Evidence

The sections above were checked against current source files, not only Markdown prose. This matters because the source has evolved past the original contract in areas such as BIOS, persistence versioning, app IDs, and media/game apps.

Evidence source	Used for
docs/algorithms.md	Initial list of algorithm topics and intended explanations.
docs/architecture/*.md	Architecture language, flow diagrams, and pattern names.
src/os/store.tsx, useOs.ts, types.ts	Reducer, state machine, facade API, window identity, z-order, boot routing.
src/os/filesystem.ts, recovery.ts, commands.ts, network.ts, audio.ts	Pure subsystem algorithms and command/effect contracts.
src/components/apps/CalculatorApp.tsx, PaintApp.tsx, MinesweeperApp.tsx, TerminalApp.tsx	Concrete app-level algorithms and effect application.
src/components/shell/WindowFrame.tsx	Drag/resize geometry and requestAnimationFrame coalescing.
src/os/persistence.ts	Current storage key win98-portfolio.v4 and persisted fields.

Design preset: compact_reference_guide. Page geometry: US Letter, 1 inch margins, Calibri 11 pt body, fixed-width DXA tables with repeating header rows.